Bubble Sort

Date : 10 / 07 / 2025
Topic : Bubble Sort

1. Introduction

Bubble Sort is one of the simplest sorting algorithms. It works by repeatedly stepping through the list, comparing adjacent elements and swapping them if they are in the wrong order. This process is repeated until the list becomes sorted.

- Simple
- In-place
- Stable
- Comparison Sort

2. Idea / Intuition

In each pass, the largest element "bubbles up" to the end of the array. After the first pass, the largest element is at the last position. After the second pass, the second largest element is at the second last position and so on...

We keep doing this until the entire array is sorted.

3. Algorithm

1. Start
2. Repeat the following steps for $i = 0$ to $n-2$
   - Compare adjacent elements a[j] and a[j+1]
   - If a[j] > a[j+1], swap them
3. After each pass, the largest element "bubbles up" to its correct position.
4. If no swaps occur in a pass, the array is already sorted. Stop early.
5. Stop

4. Pseudocode

```
BubbleSort(arr)
n ← length(arr)
for i ← 0 to n-2          // n-1 passes
    swapped ← false
    for j ← 0 to n-i-2    // Last i elements sorted
        if arr[j] > arr[j+1]
            swap(arr[j], arr[j+1])
            swapped ← true
    if swapped = false
        break             // Array is sorted
```

5. Example Dry Run

Let the array be : `5, 1, 4, 2, 8`

We will see how the array changes after each pass.

Pass 1 :

5	1	4	2	8	5 > 1 → swap
1	5	4	2	8	5 > 4 → swap
1	4	5	2	8	5 > 2 → swap
1	4	2	5	8	5 < 8 → no swap

After Pass 1 : [1, 4, 2, 5, 8] (8 is in place)

6. Key Observations

* After each pass, the largest unsorted element is placed at the end.
* After i-th pass, the last i elements are in their correct positions.
* If no swaps happen in a pass, the array is already sorted.
* Bubble Sort is stable and in-place.

Simple but powerful – Bubble Sort is the building block for understanding sorting algorithms! ♥

Bubble Sort - Page 2

Date : 10 / 07 / 2025
Topic : Bubble Sort

1. Example Dry Run (continued)

Starting array : [5 , 1 , 4 , 2 , 8]

Pass 2 :

1	4	5	2	8	4 > 1 → no swap
1	4	5	2	8	5 > 4 → no swap
1	4	2	5	8	5 > 2 → swap
1	4	2	5	8	5 < 8 → no swap

After Pass 2 : [1 , 4 , 2 , 5 , 8] (Last 2 elements are at correct place)

Pass 3 :

1	4	2	5	8	4 > 1 → no swap
1	4	2	5	8	4 > 2 → swap
1	2	4	5	8	4 > 2 → no swap

After Pass 3 : [1 , 2 , 4 , 5 , 8] (Last 3 elements are at correct place)

Pass 4 :

1	2	4	5	8	2 > 1 → no swap 2 < 4 → no swap

After Pass 4 : [1 , 2 , 4 , 5 , 8] (Array is sorted)

2. Optimized Bubble Sort

If in a pass no two elements are swapped, it means the array is already sorted. So we can stop early.

```
for i ← 0 to n-2
    swapped ← false
    for j ← 0 to n-i-2
        if arr[j] > arr[j+1]
            swap(arr[j], arr[j+1])
            swapped ← true
    if swapped = false
        break
```

Why Bubble Sort?

- Easy to understand and implement.
- Works well for small or nearly sorted arrays.
- Used in situations where simplicity is preferred over performance.

3. Actual Code (C)

```c
#include <stdio.h>
void bubbleSort(int arr[], int n) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        int swapped = 0;    // optimization
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0) break;  // no swaps → already sorted
    }
}
int main() {
    int arr[] = {5, 1, 4, 2, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Time Complexity
Best Case → O(n)
Average Case → O(n²)
Worst Case → O(n²)
Space Complexity
→ O(1)

4. Complexity

- Best Case (Already Sorted) : $O(n)$
- Average Case : $O(n^2)$
- Worst Case (Reverse Sorted) : $O(n^2)$
- Space Complexity : $O(1)$

5. Advantages

- ✓ Simple and easy to implement.
- ✓ In-place sorting (uses O(1) extra space).
- ✓ Stable algorithm.
- ✓ Works well for small or nearly sorted data.

6. Disadvantages

- × Not efficient for large datasets.
- × Many swaps are required.
- × Time complexity is $O(n^2)$ in average and worst case.

7. Important Points (For Exams/Interviews)

- Bubble Sort repeatedly swaps adjacent elements if they are in wrong order.
- After each pass, the largest element "bubbles up" to its correct position.
- It is a stable and in-place sorting algorithm.
- Time complexity is $O(n^2)$ except for best case.

8. Practice Questions

1. Perform Bubble Sort on : [7, 3, 5, 2, 9]
2. What is the time complexity of Bubble Sort?
3. Modify Bubble Sort to sort in descending order.

"Simplicity is the soul of efficiency." ♥

Common Mistakes

- Forgetting to reduce the inner loop range (n-i-1).
- Not initializing swapped = 0 in optimized version.
- Confusing swap conditions.